CONFIDENTIAL - ARCHITECTURE DOCUMENT

Customer-Application-Name

Solution architecture design

Prepared for
Customer-A

Prepared by
Beyond

Version
1.0 - May 2026

Contents

Executive summary

Customer-A asked Beyond to deliver a cloud-native, multi-tenant platform - referred to here as the Customer-Application-Name (CAN) system - that supports the end-to-end allocation, valuation, and disposal workflow of property portfolios. This document records the target architecture, the principles that shaped it, and the route by which it will reach production.

The architecture is event-driven, microservices-based, and deployed on Azure Kubernetes Service. It uses Command Query Responsibility Segregation (CQRS) and event sourcing for the core portfolio domain, a Universal Layer of Interoperability (ULI) for asynchronous messaging, and Citus-distributed PostgreSQL for the operational data store.

Authentication is delegated to Azure Active Directory with B2B guest support; the Istio service mesh enforces mutual TLS and per-service authorisation policies inside the cluster.

The recommendation is to proceed with the design as described. Risks - principally the operational maturity required for Axon Server and Citus - are identified together with mitigations.

AT A GLANCE

Cloud: Azure. Cluster: AKS + Istio. Backend: Java 21 + Spring Boot 3.2 + Axon Framework. Storage: PostgreSQL 15 + Citus 12. Messaging: NATS JetStream + Azure Event Hubs. Identity: Azure AD with B2B and MFA.

Background and engagement objectives

Business context

Customer-A operates a regulated property finance business. When a loan facility falls into default, properties securing that loan must be valued, marketed, and re-allocated to brokers and prospective buyers. The existing process is run on a mixture of spreadsheets, email, and a third-party Virtual Data Room (VDR), and is consequently slow, hard to audit, and difficult to scale across simultaneous portfolios.

Engagement objectives

The engagement set three objectives. First, capture the as-is state and target operating model. Second, design a target architecture that supports the workflow with auditability, observability, and security suitable for a regulated lender. Third, produce a buildable artefact - this document and its accompanying codebase - that the joint Customer-A and Beyond delivery team can take into implementation.

Deliverables

- A target architecture document - this document.
- A reference implementation across infrastructure as code, Kubernetes manifests, the Spring Boot backend, the database, and the ULI messaging layer.
- A skill agent pair - one for coding, one for code review - that codifies the standards in this document into the team's software development lifecycle.

Scope

In scope

- Portfolio creation, property allocation, valuation submission, broker assignment, and portfolio closure.
- VDR ingestion of supporting documents with metadata extraction.
- User and company management, including external broker and valuer onboarding via Azure AD B2B.
- Search across portfolios, properties, and documents.
- Centralised audit, logging, metrics, and tracing.

Out of scope

- Origination of new loans (handled upstream).
- Settlement and payments (handled by the existing finance system).
- Marketing automation to end-buyers.

Findings

Three findings from the discovery phase shape the design.

Finding 1 - manual handoffs dominate

More than seventy per cent of the elapsed time between a property entering the portfolio and a broker receiving a marketing pack is spent in human handoffs. The technical implication is that the architecture must prioritise asynchronous, event-driven coordination so that handoffs can be automated where rules permit and tracked where they cannot.

Finding 2 - audit is reconstructed after the fact

Today the audit trail is rebuilt at the end of each portfolio by stitching together emails, spreadsheet revisions, and VDR access logs. The implication is that the system must record an immutable, queryable event log as a first-class output of every state change. Event sourcing is therefore not a stylistic preference; it is a regulatory requirement.

Finding 3 - external participants must be onboarded quickly and securely

Brokers and valuers are external organisations. They must be granted access within hours, not days, and that access must be revoked just as quickly. Azure AD B2B with conditional access addresses this: invitations are sent in seconds, multi-factor authentication is enforced at the identity provider, and access tokens are short-lived. Application authorisation is then layered on top via OAuth scope claims.

Recommendations

1. Adopt an event-driven, CQRS-and-event-sourcing architecture for the portfolio bounded context, using AxonFramework on Spring Boot. This gives the audit guarantee and the read/write decoupling the workflow needs.
2. Use NATS JetStream as the in-cluster message bus, wrapped in a Universal Layer of Interoperability (ULI) so every cross-service message - whether on NATS, Azure Event Hubs, or the Axon bus - shares one envelope, one correlation strategy, and one dead-letter discipline.
3. Run on AKS with Istio in mTLS STRICT mode. Identify every workload with Azure Workload Identity and pull secrets through the Key Vault Container Storage Interface (CSI) driver - never as plain Kubernetes Secrets.
4. Persist application state in a Citus-distributed PostgreSQL cluster. Co-locate by `portfolio_id` so all per-portfolio joins are local; use reference tables for small lookup data; distribute the audit log by `correlation_id`.
5. Treat Azure AD as the authoritative identity provider. Validate JWTs twice - once at the Istio ingress gateway, once at the Spring Boot resource server - and authorise inside services with method-level `@PreAuthorize` against OAuth scope claims.

Architecture guidelines and principles

Eight principles govern every design decision in the rest of this document. Where the principles conflict in a specific situation, the order they appear in expresses the priority.

1. Auditability is a first-class output

Every state change is recorded as an immutable domain event. The event store is the system of record; read models are derived. Where event sourcing is not appropriate (for example, simple lookup tables), an append-only audit log is maintained instead.

2. Asynchronous by default, synchronous by exception

Cross-service communication is asynchronous through the ULI. Synchronous calls are reserved for read-side queries and explicit user requests where the user is waiting for an answer. This protects the system from cascading failures and makes back-pressure tractable.

3. Identity is delegated, authorisation is local

The system never stores passwords. Azure AD issues tokens. Each service validates the token and decides authorisation locally based on OAuth scope claims and, where relevant, role assignments fetched from the user-management service.

4. Secrets are never in code or container images

Secrets are stored in Azure Key Vault and mounted via the CSI driver under a workload identity. Connection strings, signing keys, and API tokens are rotated by re-deploying the workload, not by re-building images.

5. Every request carries a correlation ID

A correlation ID is minted at the edge if absent, propagated via the `X-Correlation-Id` header on synchronous calls, attached to every ULI envelope, written into the structured log MDC, and emitted on the response. One correlation ID maps a user action to every log line, span, and event it produced.

6. Logs are structured and centralised

All logs are JSON to standard output. Azure Monitor ingests stdout, parses each line, and surfaces fields as columns. Free-text-only logging is forbidden. Personally identifiable information is never logged.

7. Defence in depth

Edge - Azure AD authentication + WAF. Mesh - mTLS STRICT and authorisation policies. Service - JWT validation and method-level authorisation. Data - private endpoints, Citus row-level co-location boundaries, Postgres role separation. The breakdown of any one layer must not compromise the system.

8. Cost-aware by design

Premium tiers are used only where they materially improve security or availability (Key Vault, ACR Premium, AKS uptime SLA). Storage uses lifecycle rules to move cold data to cool tiers. Non-production environments scale to zero overnight.

Standards

Application standards

Java 21 with virtual threads where contention is observed. Spring Boot 3.2 for runtime, AxonFramework 4.10 for CQRS and event sourcing. All public APIs are documented with OpenAPI 3.1 (REST) or schema-first GraphQL (the BFF). Dates are ISO-8601, identifiers are UUIDv4, money values are stored as `NUMERIC(18,2)` and never as floats.

Infrastructure standards

Infrastructure is defined in Terraform 1.7 against the AzureRM 3.x provider. State is held in an Azure Storage account with state locking. Terraform modules are versioned and tagged. Kubernetes manifests are organised by Kustomize. Container images are built with Jib (no Dockerfiles) and signed with Cosign before being pushed to a Premium ACR with vulnerability scanning enabled.

Data standards

PostgreSQL 15 with the Citus extension. Per-bounded-context schemas inside a single application database. Migrations are managed by Flyway from each owning service. Partitioning by `portfolio_id` is the co-location key for distributed tables. The audit log is append-only with database-enforced triggers blocking UPDATE and DELETE.

Security standards

TLS 1.2 minimum on every external endpoint, mutual TLS in STRICT mode inside the mesh. OAuth 2.0 with the OpenID Connect profile, AAD-issued JWTs validated twice. Role-based access control at the database. Network policies default-deny. Container images run as non-root with a read-only root filesystem and dropped capabilities.

CI/CD standards

Trunk-based development. Pull requests must pass unit tests, integration tests, SAST, dependency vulnerability scans, container scanning, and a quality gate before merge. Production releases require two-person approval and follow a canary rollout. Every released image carries a Software Bill of Materials (SBOM) in CycloneDX format.

Languages and frameworks

Tier	Language / framework	Rationale

Web UI	TypeScript, React 18, Vite	Familiar, accessible build tooling, large component ecosystem
BFF	Java 21, Spring Boot, GraphQL	Single GraphQL endpoint for the UI, type-safe contracts
Microservices	Java 21, Spring Boot, Axon	Mature CQRS and event-sourcing tooling
VDR ingestion	Apache Tika, POI, PDFBox	Industry standard for office and PDF text extraction
Messaging	NATS JetStream + Event Hubs	In-cluster speed plus durable, geo-redundant retention
Data	PostgreSQL 15 + Citus 12	Familiar SQL surface with horizontal scale
IaC	Terraform 1.7	Declarative, well supported on Azure
CI/CD	GitHub Actions, ArgoCD	GitOps-friendly, hosted, auditable

TABLE 1 - LANGUAGES, FRAMEWORKS, AND THE RATIONALE FOR EACH.

Patterns

Application authentication – Azure AD B2B with MFA

Internal users sign in through their Customer-A AD account, which is synchronised into Azure AD. External users (brokers, valuers) are invited as B2B guests through the Microsoft Graph invitation API. Conditional access enforces multi-factor authentication for any access from outside the corporate network. The flow is shown in Figure 1.

FIGURE 1 - AZURE AD B2B AUTHENTICATION AND MFA FLOW.

Centralised logging

Every pod writes JSON logs to standard output. The Azure Monitor agent on each AKS node scrapes stdout, parses each line, and writes it to Log Analytics. Application Insights derives traces and metrics from the same source. Operators do not log into pods to retrieve logs.

Correlate requests with a unique correlationId

A correlation ID is generated at the Istio ingress if the request did not carry one. The Spring Boot `CorrelationIdFilter` puts it into the SLF4J Mapped Diagnostic Context (MDC). Outbound `WebClient` calls and ULI envelopes propagate it. Every log line and span therefore carries the same value, and a single Kusto query joins them.

Structure your log data

Logs are emitted via the Logstash JSON encoder for Logback. The structure carries timestamp, level, logger, thread, message, MDC fields (correlation, trace, span), service name, environment, and exception detail when present. Free-text logs are rejected by the linter.

Choreography over orchestration where possible

Services react to events on the ULI rather than being orchestrated by a central workflow engine. Where orchestration is required - the property allocation lifecycle is the canonical case - it is implemented as an Axon Saga in the workflow service, which itself reacts to events and emits commands.

Outbox / inbox / dead-letter

Producers write to a database outbox in the same transaction as the state change, then publish from the outbox. Consumers record the message identifier in an inbox before applying side effects, giving idempotency. Poison messages are parked on `uli.dlq` after the configured retry budget.

Current stack

The existing process relies on three categories of tool: a Customer-A-hosted Microsoft Office estate (Excel, Outlook, SharePoint) for portfolio tracking; a third-party VDR for document distribution; and a custom .NET internal tool for valuation entry, backed by a SQL Server database. The pain points are familiar - data integrity drift between the three, no end-to-end audit trail, and slow external onboarding - and they are the proximate motivation for this engagement.

Proposed new stack

Application architecture view

The system is structured as a small number of bounded contexts, each owning its data, communicating asynchronously through the ULI, and presenting a synchronous read API to the BFF. Figure 2 shows the high-level shape.

FIGURE 2 - HIGH-LEVEL ARCHITECTURE. USERS AUTHENTICATE AGAINST AZURE AD; TRAFFIC ENTERS AKS THROUGH ISTIO; MICROSERVICES COORDINATE VIA NATS JETSTREAM AND AXON; DATA LIVES IN CITUS POSTGRESQL.

Conceptual view

Five bounded contexts compose the system. `user-auth` owns identity, role assignment, and the Microsoft Graph integration that creates B2B guests. `property-allocation` owns the portfolio aggregate - the heart of the domain - and is implemented with CQRS and event sourcing. `vdr-ingestion` extracts text and metadata from documents uploaded to the Virtual Data Room. `workflow` orchestrates the multi-step allocation saga. `search` indexes events into a full-text searchable projection. The BFF aggregates these for the React UI.

FIGURE 3 - WRITE SIDE AND READ SIDE OF THE PROPERTY-ALLOCATION CONTEXT. COMMANDS PRODUCE EVENTS; EVENTS FEED PROJECTIONS, SAGAS, AND THE ULI BRIDGE.

Portfolio data model

The portfolio domain is modelled around six core entities: `portfolio`, `property`, `unit`, `property_valuation`, `broker_allocation`, and `property_journal`. Each portfolio aggregates one or more properties; each property comprises one or more units; valuations and broker allocations attach to properties; the journal records every state change for the portfolio as a whole. Tables are distributed by `portfolio_id` so all per-portfolio joins remain on a single Citus worker.

Entity	Distribution	Purpose
portfolio	Distributed by id	Top-level aggregate root
property	Co-located on portfolio	One property in the portfolio
unit	Reference table	Lease unit under a property
property_valuation	Co-located on portfolio	Latest valuation per property
broker_allocation	Co-located on portfolio	Which broker markets which property
property_journal	Co-located on portfolio	Append-only event log
company / app_user	Reference table	Small lookup; replicated to every worker
audit.event_log	Distributed by correlationId	All events of one logical request co-locate

TABLE 2 - PORTFOLIO DATA MODEL AND THE CITUS DISTRIBUTION STRATEGY FOR EACH ENTITY.

Universal Layer of Interoperability

The ULI is an envelope-plus-protocol layer that sits over NATS JetStream and Azure Event Hubs. Every cross-service message - whether a domain event, an asynchronous command, or a query - is wrapped in a UliEnvelope record and published on a hierarchical subject of the form `uli.<kind>.<context>.<Type>`. Consumers are durable, dedupe via an inbox table, and park poison messages on the dead-letter stream. Figure 4 shows the topology.

FIGURE 4 - UNIVERSAL LAYER OF INTEROPERABILITY. PRODUCERS WRAP DOMAIN EVENTS IN THE ULI ENVELOPE AND PUBLISH TO NATS SUBJECTS; DURABLE CONSUMERS PROCESS EVENTS WITH AT-LEAST-ONCE SEMANTICS; THE EVENT HUBS BRIDGE REPLICATES TO DURABLE STORAGE.

Cluster and network topology

Figure 5 shows the AKS network topology. Public traffic enters through Application Gateway with WAF, then through an internal Istio ingress gateway. AKS, AKS-managed resources, and PaaS data services sit in a hub-and-spoke virtual network with private endpoints for PostgreSQL, Storage, Key Vault, and ACR. Egress is forced through Azure Firewall with an FQDN allow-list.

FIGURE 5 - AKS NETWORK TOPOLOGY AND SECURITY BOUNDARIES.

Route to live

CI

Continuous integration runs in GitHub Actions. Each pull request triggers `mvn verify`, SonarQube analysis, Trivy and Snyk scans, container image build with Jib, SBOM generation, and Cosign signing. The pipeline publishes signed images to ACR on merge to `main`.

Environments

- Dev - auto-deployed on every successful build of `main`. Smoke tests run after deploy.
- UAT - tag-promoted from dev. Acceptance tests and a small performance run.
- Pre-prod - prod-like data, full load test, disaster recovery drill at the start of every quarter.
- Prod - manual approval gate. Canary rollout with progressive traffic shift; automatic rollback on SLO breach.

Testing

Unit tests use JUnit 5 and the Axon test fixture for aggregate behaviour. Integration tests use Testcontainers for PostgreSQL, NATS, and Azure Storage. Contract tests use Pact between the BFF and downstream services. End-to-end tests run after deploy to the UAT environment using Playwright. Coverage threshold is 80% line, 70% branch.

On-boarding

A new developer should be productive within one day. The onboarding pipeline is documented in the repo `README.md`: install JDK 21, Maven 3.9, Docker, and the Azure CLI; run `docker compose -f compose.dev.yml up -d` to bring up Postgres, NATS, and Axon Server; run any service with `mvn -pl <service> spring-boot:run`. There is no proprietary tooling required.

Support integration

Operational alerts are delivered to a single Microsoft Teams channel through Azure Monitor action groups. Pager duty rotation is held by the platform team. Run-books are in the repo, one per service, listing the top five symptoms and their first-line response. Every incident produces an after-action review filed against the architecture decision record (ADR) folder if the response changes the design.

Route-to-live process and minimum requirements

Figure 6 shows the route to live. The minimum requirements list is the merge gate: no PR can land on `main` without all of them green.

FIGURE 6 - ROUTE TO LIVE. PULL REQUEST OPENS; CI RUNS; IMAGE SIGNED AND PUBLISHED; ARGOCD RECONCILES TO DEV, UAT, PRE-PROD, AND FINALLY PROD WITH A MANUAL GATE.

Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Axon Server operational maturity	Medium	High	Three-node cluster, separate event store DB, weekly restore drill from snapshots
Citus shard skew	Medium	Medium	Monitor `pg_dist_shard_placement` size; resharding playbook; portfolio_id chosen because it spreads naturally
JWT validation drift between Istio and Spring	Low	High	Single source of truth for issuer and audience in Helm values; integration test asserts both reject the same invalid token
NATS JetStream persistence loss	Low	High	3 replicas + Event Hubs bridge + outbox in PG provides three independent durability tiers
Azure AD outage	Low	High	Token caching at the BFF for the duration of token validity; degraded read-only mode with cached authorisation

TABLE 3 - TOP ARCHITECTURAL RISKS AND THEIR MITIGATIONS.

Conclusion

Acting on the recommendations in this document gives Customer-A a CAN system that is auditable by construction, scales horizontally inside the portfolio domain, and onboards external users in minutes rather than days. The reference implementation accompanying this document is structured to be lifted directly into Customer-A's Azure subscription with the customary tenant-specific substitutions. The next step is to schedule the kick-off and confirm the cut-over date for the first pilot portfolio.

About Beyond

Beyond is a global technology group, headquartered in London. It deploys experts and frontier technology, including AI, to help organisations thrive through change. With over 600 professionals - more than 75% hands-on technical talent - across Europe, North America, and Asia, Beyond is backed by Marlin Equity Partners. Core value propositions: building modern, scalable infrastructure; creating digital experiences that convert; and establishing security, governance, and compliance systems.